

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

Desarrollo Web · II semestre 2026

Informe de OpenSSL y Sodium

Ricardo Caballero

Grupo 1S3122

ricardo.caballero5@utp.ac.pa

Docente: Ing. Irina Fong

29 de octubre de 2026

1. Introducción y objetivos

En este laboratorio se prueban varias formas de proteger datos con PHP. Se usan los ejemplos de la profesora y se agregan formularios para escribir mensajes y ver los resultados. Con OpenSSL se crean firmas, solicitudes de certificado y certificados de prueba. También se cifra un mensaje y se compara el resultado con Sodium.

El objetivo es entender cómo funcionan estas herramientas al usarlas. Para eso se generan claves RSA de 2048 bits, se revisan firmas con SHA-256, se crea un certificado de prueba y se recupera un mensaje cifrado. También se cambian los datos a propósito para comprobar cuándo falla la verificación.

Las pruebas se hicieron en el equipo local con PHP 8.3.36, OpenSSL 3.0.22 y Sodium 1.0.22, usando datos de ejemplo. Los certificados creados aquí sirven para practicar; no reemplazan un certificado emitido por una autoridad de confianza.

Código y evidencias: https://github.com/ricardo-caballero2663/ricardo-caballero-laravel

Componente	Implementación y resultado
firma7	Firma RSA/SHA-256; prueba válida y prueba con mensaje alterado.
firmaOtra	Formulario de identidad de prueba; genera CSR y certificado autofirmado.
FirmaMensaje	Usa la clave privada y el certificado de firmaOtra en la misma sesión.
Encriptacion	Reproduce AES-128-CBC y añade una comparación autenticada AES-256-GCM.
Sodium	Secretbox: recuperación correcta y rechazo de cifrado manipulado.

2. Cómo se preparó el laboratorio

Se tomaron como referencia firma7.php, firmaOtra.php, FirmarMesaje.php y Encriptacion.php. El código se organizó en crypto.php y public/index.php para compartir las funciones y mostrar los formularios. En el menú se usa FirmaMensaje, corrigiendo el nombre FirmarMesaje del ejemplo original.

La configuración se carga desde __DIR__/openssl.cnf, para que no dependa de una carpeta fija del equipo. La CSR es una solicitud y empieza con BEGIN CERTIFICATE REQUEST; el certificado empieza con BEGIN CERTIFICATE. La aplicación genera claves nuevas en lugar de usar las publicadas en el ejemplo.

Los formularios revisan que el mensaje no esté vacío y que no sea demasiado largo. También protegen la salida HTML y usan un token CSRF. Las claves de firmaOtra se guardan en la sesión mientras se hace la prueba. Las claves que aparecen en las capturas son de ejemplo y no se usan fuera de este laboratorio.

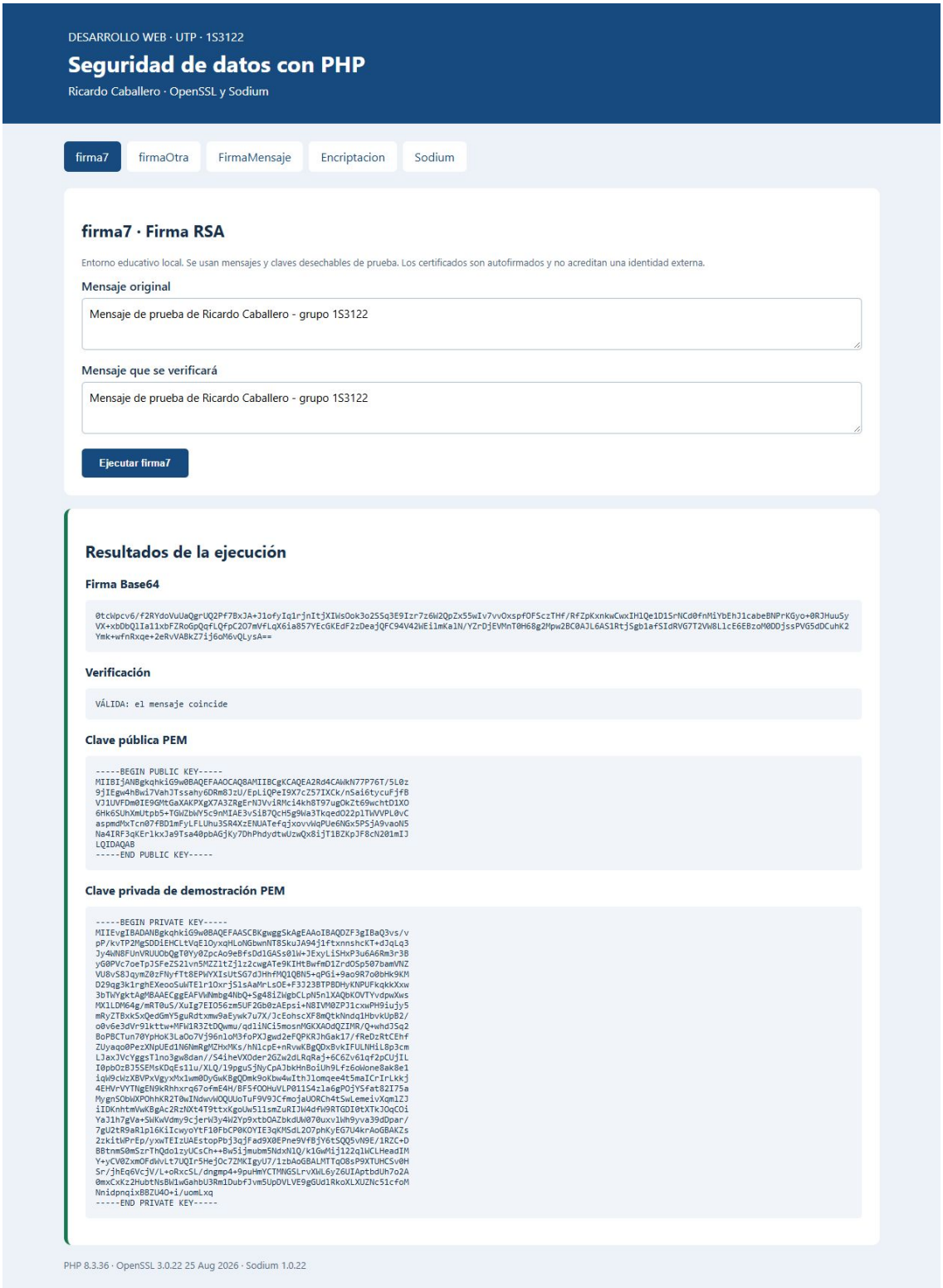
Con AES-CBC se puede cifrar y recuperar el mensaje, pero eso por sí solo no demuestra que nadie lo cambió. Por eso se agregó una prueba con AES-GCM, un nonce de 12 bytes y una etiqueta de autenticación. Al cambiar un byte del cifrado, la verificación falla. Base64 solo permite mostrar los datos como texto: no los protege.

Para guardar contraseñas se debe usar un hash adecuado, como Argon2id, en vez de cifrarlas de forma reversible. Las claves deben guardarse aparte de los datos cifrados. Esta práctica usa HTTP en el equipo local; una aplicación publicada necesita HTTPS para proteger la conexión.

Las 11 pruebas automáticas pasaron. Las seis capturas muestran la aplicación en funcionamiento. También se comprobó que el certificado corresponda a la clave privada y que no se acepten mensajes vacíos.

3. Evidencia: firma7

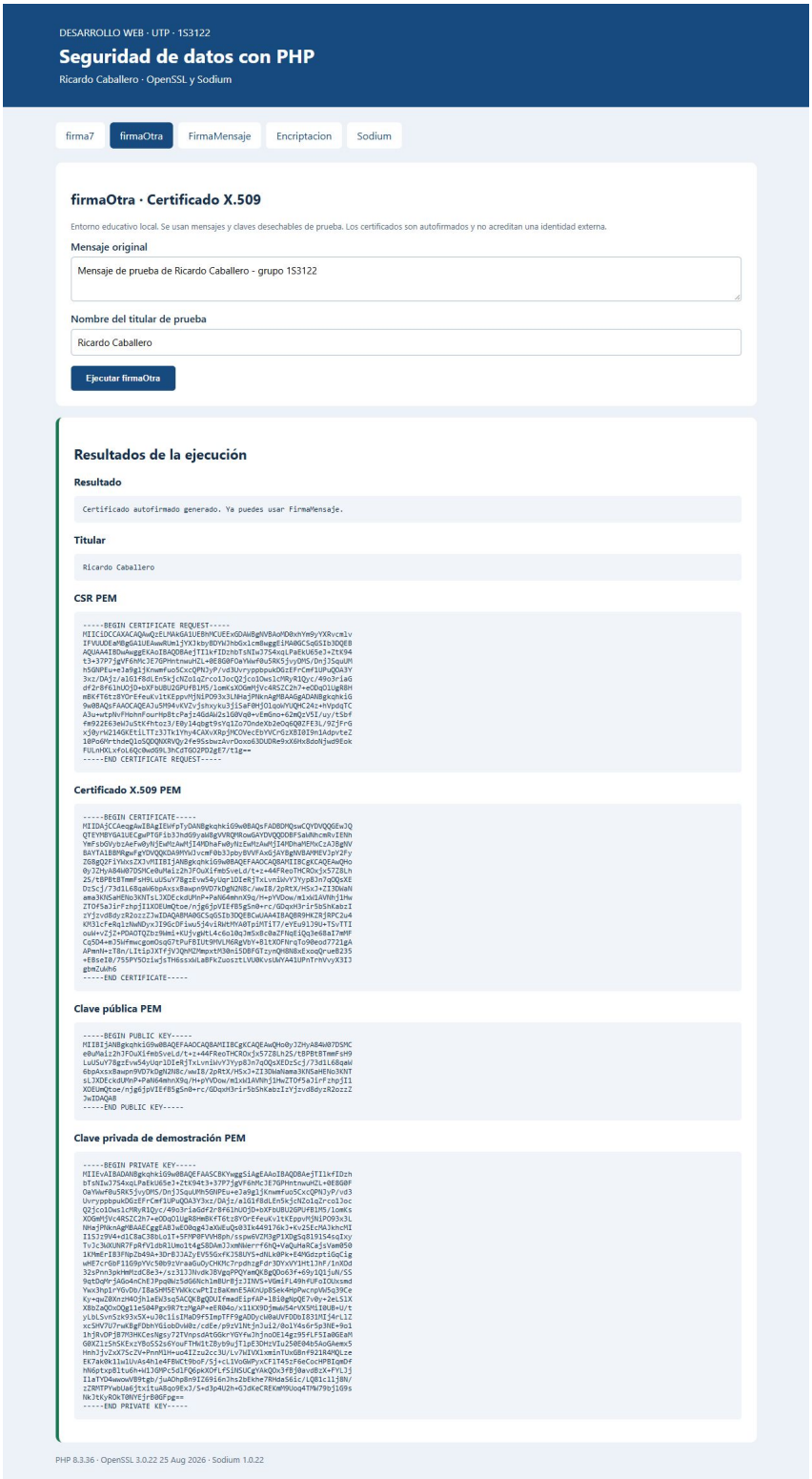
El formulario recibe dos mensajes iguales. La verificación devuelve VÁLIDA y se muestran firma y par de claves generado.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

4. Evidencia: firmaOtra

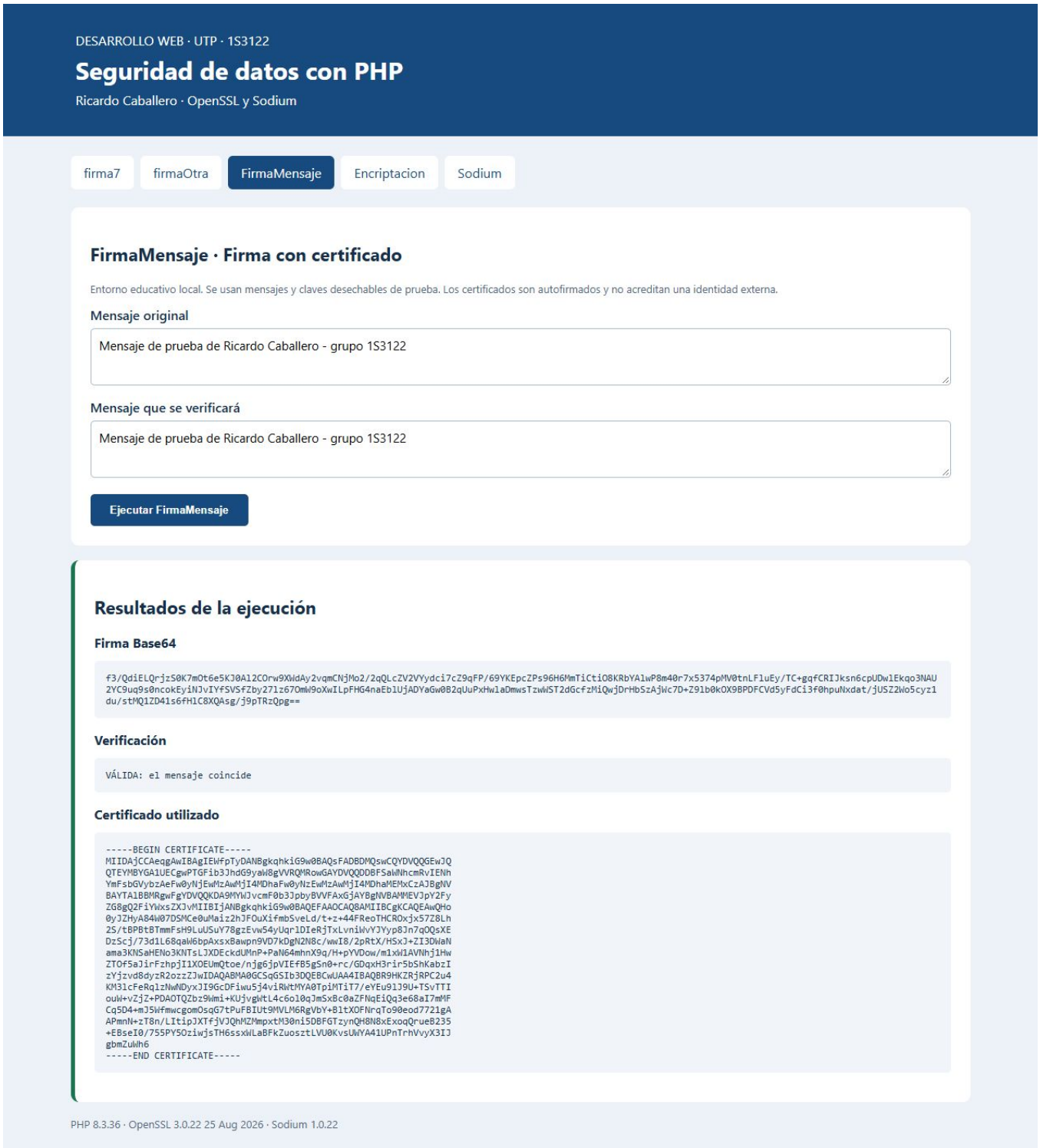
El nombre ingresado se incorpora al certificado de prueba. La pantalla muestra la CSR, el X.509 y las claves generadas.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

5. Evidencia: FirmaMensaje

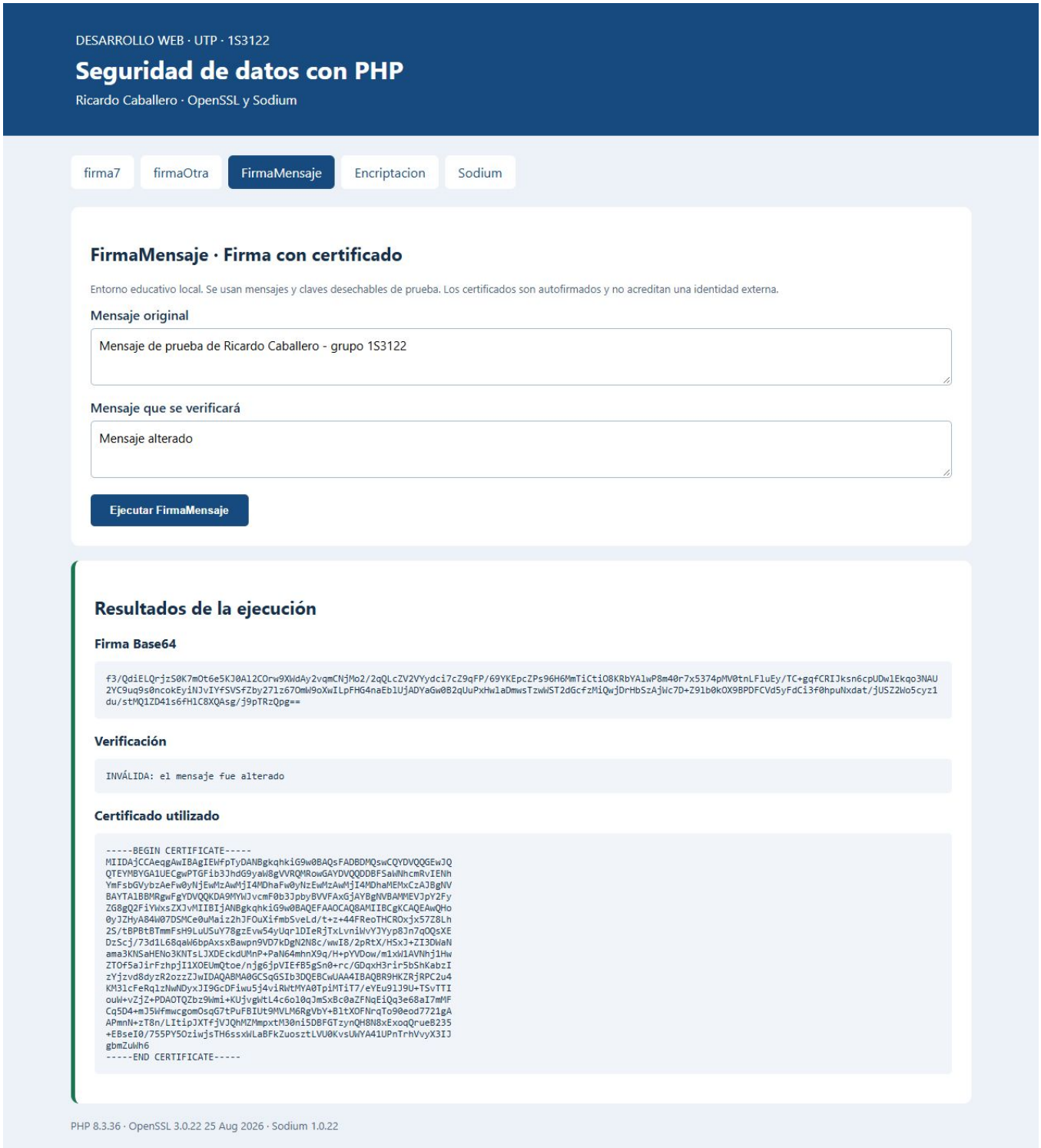
La firma se verifica con el certificado creado anteriormente en la misma sesión. Resultado: VÁLIDA.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

6. Evidencia: alteración

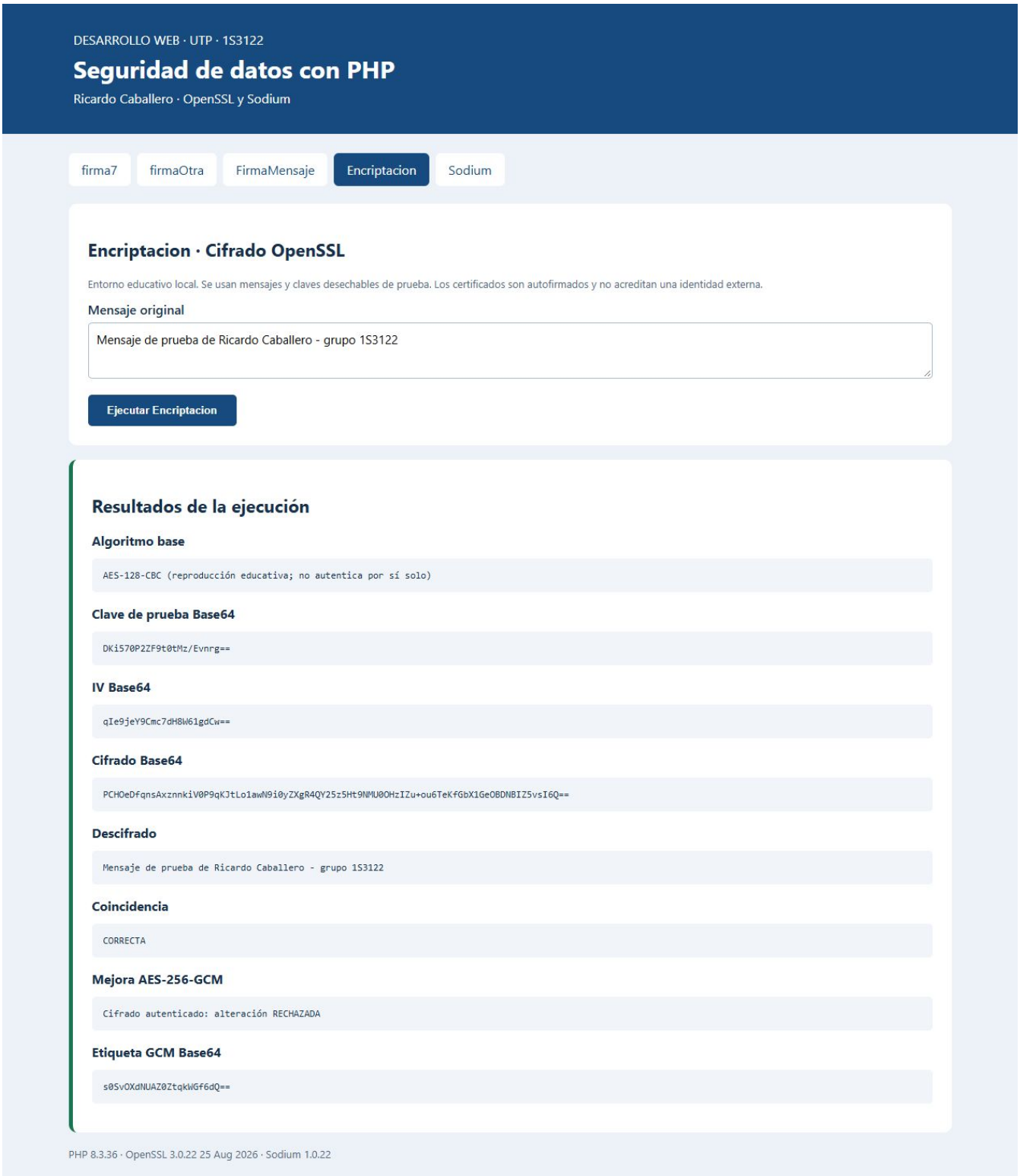
Al sustituir el texto de verificación por Mensaje alterado, la firma pasa a INVÁLIDA. La firma no oculta el mensaje; permite detectar cambios.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

7. Evidencia: cifrado OpenSSL

AES-CBC recupera el texto original. La comparación adicional con AES-GCM rechaza el cifrado alterado.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

8. Evidencia: Sodium

Secretbox recupera el mensaje y rechaza la alteración. El nonce y el cifrado se muestran en Base64 para facilitar la inspección.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

9. Funciones usadas

Función	openssl_pkey_new / openssl_pkey_export
Propósito	Generar RSA y exportar PEM
Parámetros	Arreglo con RSA, 2048 bits y archivo openssl.cnf; clave, variable PEM por referencia, contraseña opcional.
Retorno	Objeto OpenSSLAsymmetricKey o false; export devuelve bool.
Proceso	Genera dos claves relacionadas. La privada se conserva solo en la sesión de demostración.

Función	openssl_pkey_get_details / openssl_pkey_get_private
Propósito	Obtener clave pública o cargar privada
Parámetros	Objeto de clave; cadena PEM privada.
Retorno	Arreglo con key/bits/type o false; objeto de clave o false.
Proceso	Permite separar verificación pública y firma privada.

Función	openssl_sign
Propósito	Firmar mensaje
Parámetros	Datos, firma binaria por referencia, clave privada y OPENSSL_ALGO_SHA256.
Retorno	bool; firma se escribe en el parámetro de salida.
Proceso	RSA firma un resumen SHA-256. No cifra el texto ni lo vuelve confidencial.

Funciones usadas (continuación)

Función	openssl_verify
Propósito	Verificar firma
Parámetros	Mensaje, firma binaria, clave pública o certificado, algoritmo SHA-256.
Retorno	1 válida; 0 inválida; -1 o false si hay error.
Proceso	Comprueba correspondencia entre mensaje y firma. Se distinguen alteración y fallo técnico.

Función	openssl_csr_new / openssl_csr_export
Propósito	Crear solicitud de certificado
Parámetros	DN (país, organización, nombre), clave privada, configuración; CSR y salida PEM.
Retorno	OpenSSLCertificateSigningRequest o false; export devuelve bool.
Proceso	La CSR contiene identidad declarada y clave pública, firmadas con la privada.

Función	openssl_csr_sign / openssl_x509_export
Propósito	Emitir certificado de prueba
Parámetros	CSR, certificado emisor null, clave privada, 365 días, configuración, serie; certificado y salida PEM.
Retorno	OpenSSLCertificate o false; export devuelve bool.
Proceso	Se crea un X.509 autofirmado. La confianza externa requiere una autoridad o mecanismo de confianza adicional.

Funciones usadas (continuación)

Función	openssl_encrypt / openssl_decrypt
Propósito	Cifrar y recuperar datos
Parámetros	Texto/cifrado, algoritmo, clave, opciones, IV; en GCM etiqueta y datos asociados opcionales.
Retorno	Cadena binaria con OPENSSL_RAW_DATA o false.
Proceso	CBC usa relleno y exige protección de integridad adicional; GCM incorpora etiqueta autenticadora.

Función	random_bytes / openssl_cipher_iv_length
Propósito	Generar aleatoriedad y medir IV
Parámetros	Cantidad de bytes; nombre del cifrador.
Retorno	Cadena aleatoria; longitud entera o false.
Proceso	Evita IV constantes. La clave y el IV se generan con tamaños adecuados al algoritmo.

Función	sodium_crypto_secretbox_keygen
Propósito	Generar clave simétrica Sodium
Parámetros	Sin parámetros.
Retorno	32 bytes aleatorios.
Proceso	Produce la clave requerida por secretbox; no se reutiliza para otros fines.

Funciones usadas (continuación)

Función	sodium_crypto_secretbox
Propósito	Cifrado autenticado
Parámetros	Mensaje, nonce único de 24 bytes y clave de 32 bytes.
Retorno	Cadena con cifrado y autenticador.
Proceso	XSalsa20 cifra y Poly1305 autentica. El nonce puede acompañar al cifrado; la clave debe permanecer secreta.

Función	sodium_crypto_secretbox_open / sodium_memzero
Propósito	Verificar, descifrar y limpiar clave
Parámetros	Cifrado, nonce y clave; variable sensible por referencia.
Retorno	Texto o false si falla autenticación; memzero devuelve void.
Proceso	No se usa el texto si falla la autenticación. La limpieza reduce la permanencia de la clave en memoria.

Función	base64_encode / base64_decode
Propósito	Representar binarios como texto
Parámetros	Cadena binaria; cadena Base64 y strict=true al decodificar entradas externas.
Retorno	Cadena; decode puede devolver false.
Proceso	Base64 es codificación, no cifrado. Se usa para mostrar firmas, nonce y resultados.

10. Investigación: Libsodium

Libsodium reúne funciones de cifrado que son más sencillas de usar. PHP incluye soporte para Sodium desde la versión 7.2, pero hay que revisar que la extensión esté habilitada. Sus funciones ayudan a crear claves del tamaño correcto. Aun así, hay que guardar bien la clave y no repetir el nonce al usar esa misma clave.

Operación	Algoritmos y utilidad
Cifrado simétrico	secretbox combina XSalsa20 y Poly1305. Las interfaces AEAD permiten XChaCha20-Poly1305 y datos asociados.
Cifrado asimétrico	crypto_box utiliza claves públicas/privadas para compartir mensajes autenticados; se apoya en Curve25519 y primitivas simétricas.
Firmas	Ed25519 permite firmar y verificar mensajes con claves separadas. Una firma no cifra el contenido.
Hashing	BLAKE2b para hashing general; Argon2id para derivación y protección de contraseñas con costo de memoria.
Rendimiento	Las primitivas están implementadas en una biblioteca nativa y evitan diseñar combinaciones manuales. La velocidad relativa depende de CPU, tamaño y operación; no se realizó un benchmark comparativo.

En este ejemplo se crea una clave de 32 bytes y un nonce aleatorio de 24 bytes. secretbox cifra el mensaje y agrega la información necesaria para comprobar si fue modificado. secretbox_open devuelve el texto si todo está bien, o false si los datos fueron alterados. Se probaron los dos casos.

OpenSSL sirve cuando se necesitan firmas RSA, solicitudes CSR o certificados X.509. Sodium facilita otras tareas de cifrado dentro de una aplicación. En ambos casos hay que cuidar las claves: elegir una buena herramienta no basta si las claves quedan expuestas.

11. Ejemplo ejecutable y conclusiones

```
$key = sodium_crypto_secretbox_keygen();
$nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$cipher = sodium_crypto_secretbox($mensaje, $nonce, $key);
$plain = sodium_crypto_secretbox_open($cipher, $nonce, $key);
if ($plain === false) {
    throw new RuntimeException('Autenticacion fallida');
}
sodium_memzero($key);
```

El código completo incluye un formulario, mensajes de error y una prueba que modifica el cifrado. Se puede repetir la revisión con tests.php y consultar el resultado en pruebas.txt. La aplicación genera sus claves al ejecutarse.

La principal conclusión es que cifrar y firmar no hacen lo mismo. Cifrar ayuda a ocultar un mensaje; una firma permite comprobar si coincide con el que se firmó. En las pruebas, al cambiar el mensaje o el cifrado, RSA, GCM y secretbox detectaron el cambio mediante su verificación.

El certificado autofirmado fue útil para practicar, pero no demuestra por sí solo la identidad de una persona frente a otros. También quedó claro que las claves deben generarse, guardarse y renovarse con cuidado.

Para una próxima aplicación conviene guardar solo los datos necesarios, seguir la documentación y probar tanto los casos correctos como los errores. Lo más útil de este laboratorio es entender qué protege cada función y qué cosas todavía hay que cuidar.

Referencias consultadas el 29 de octubre de 2026

Repositorio de la docente - https://github.com/Salomon2514/SeguridadOpenSSL

PHP OpenSSL - https://www.php.net/manual/en/book.openssl.php

Firma digital en PHP - https://www.php.net/manual/en/function.openssl-sign.php

Cifrado en PHP - https://www.php.net/manual/en/function.openssl-encrypt.php

PHP Sodium - https://www.php.net/manual/en/book.sodium.php

Secretbox - https://doc.libsodium.org/secret-key_cryptography/secretbox

Firmas Ed25519 - https://doc.libsodium.org/public-key_cryptography/public-key_signatures

Hashing de contraseñas - https://doc.libsodium.org/password_hashing

12. Paso a paso de ejecución

1. Instalar PHP 8.2 o superior. Comprobar con php -v y php -m que estén habilitadas openssl y sodium. En Windows, habilitar extension=openssl y extension=sodium en el php.ini correspondiente y reiniciar el servidor.

2. Abrir una terminal dentro de aplicacion. El archivo openssl.cnf incluido evita depender de una ruta fija como C:\OpenSSL-Win64. No se requiere MySQL.

3. Ejecutar php -S 127.0.0.1:18085 -t public. Abrir http://127.0.0.1:18085 en el navegador. El servidor de desarrollo se limita al equipo local.

4. En firma7, escribir un mensaje en ambos campos y pulsar Ejecutar firma7. Debe aparecer VÁLIDA. Cambiar el campo de verificación y repetir para obtener INVÁLIDA.

5. En firmaOtra, escribir el nombre del titular de prueba y ejecutar. Revisar la CSR, el certificado X.509 y las claves PEM generadas. Mantener la misma sesión del navegador.

6. Entrar a FirmaMensaje y ejecutar con textos iguales. Se usa la clave y el certificado creados en el paso anterior. Cambiar el mensaje que se verificará y comprobar que la firma sea rechazada.

7. En Encriptacion, ingresar texto y ejecutar. Comparar el descifrado con el original. Revisar IV, cifrado Base64 y el rechazo de alteración en la prueba adicional AES-256-GCM.

8. En Sodium, ingresar texto y ejecutar. Revisar nonce, cifrado, recuperación del mensaje y rechazo de una modificación de un byte.

9. Ejecutar php tests.php en la carpeta aplicacion. Deben aprobarse 11 pruebas. Si sodium no está cargado, usar php -d extension=sodium tests.php tras verificar que la extensión esté instalada.

10. Cerrar el servidor con Ctrl+C. Las claves de demostración no deben reutilizarse. En producción se requieren gestión de secretos, HTTPS, controles de acceso y almacenamiento seguro.